Diet Analyzer

Javier Gutierrez Back & George Chaidemenos

2024-04-23

Table of contents

Introduction 1
Application and Usage 3
Example 4
Evaluation 6
Conclusion 7
References 8
Appendix 8

Introduction

Title: Diet Analyzer

Purpose:

We want to make it easy for people to visualise what they ate in a particular day in terms of their nutrients intake, and somehow quantify how 'balanced' their diet was. One question we would like to answer is what proportion of a person's meal was in each food group. Additionally, we would like people to be able to input to the program what they ate in the meal and have it say the amounts of each nutrient that they had. People would care about our project since it would be easier for them to find patterns in their eating habits and how to make it healthier perhaps or help them plan meals in advance.

Data:

We found our data set in the data is plural website. It is called Canadian Nutrient and it is a relational data base that provides information about food descriptions, food groups, nutrients, and a method for calculating the amounts of these nutrients for different quantities of the food. The data set contains 12 tables in a csv format which were already documented and loaded as an R package.

We will not be working with spatial data in shapefiles, or using an API to a live data source. (https://www.canada.ca/en/health-canada/services/food-nutrition/healthy-eating/nutrient-data/canadian-nutrient-file-2015-download-files.html)

Variables:

- food: type of food
- nutrient: name of the nutrient
- nutrient unit: the unit of measure (g, mg)
- nutrient quantity: nutrient quantity in nutrient unit per 100 grams of the food
- measure name: type of quantity used to measure the food
- conversion factor value: the factor by which me multiply the nutrient quantity per the measure used
- food group: the general category the group belongs to

End Product:

We would hope to create a user interactive app through shiny where the user will input what foods they had eaten in a day and the app will provide the user with an analysis of their diet for that particular day. For example, create a pie chart or other visualisation that shows what proportions of different food groups they consumed, and a detailed analysis of how much fat, protein or other specific nutrients they consumed. We would like the user to be able to specify the specific foods and quantities as closely to the ones on the data set to provide accurate data with the conversion factors. For both of these questions, we would like to compare the user's numbers with certain baseline diets, like an athlete, a bodybuilder, a health nut, keto diet, vegan, maybe gluten free, fast food enthusiast. That is, showing the food group proportions of for the corresponding diets, do some sort of calculation to see which one the user is closest too, and maybe provide sample diets or possible improvements to make the diet more balanced or to satisfy your objective diet requirements.

For the Shiny app, we will probably like to have an 'input your meal' section, where the user can type in their foods and match them (with regular expressions in the back end) to the foods in the database. From there, you will see two sections, one with the food group proportion graph, and one with your nutrient intake in descending order. We could still figure out whether we want the baseline diet comparison in the same sections, or if we want another section below those with possible changes and the comparisons.

```
#| include: false

find_diet_fit <- function(props, athlete, normal, vegan, dessertlover, carnivore, fastfood

  best_fit <- 'athlete'
  athlete_err <- sum_squared_errors(props, athlete)
```

```
  min <- athlete_err

  normal_err <- sum_squared_errors(props, normal)
  if (normal_err < min) {
    min <- normal_err
    best_fit <- 'normal'
  }

  vegan_err <- sum_squared_errors(props, vegan)
  if (vegan_err < min) {
    min <- vegan_err
    best_fit <- 'vegan'
  }

  dessertlover_err <- sum_squared_errors(props, dessertlover)
  if (dessertlover_err < min) {
    min <- dessertlover_err
    best_fit <- 'dessertlover'
  }

  carnivore_err <- sum_squared_errors(props, carnivore)
  if (carnivore_err < min) {
    min <- carnivore_err
    best_fit <- 'carnivore'
  }

  fastfoodlover_err <- sum_squared_errors(props, fastfoodlover)
  if (fastfoodlover_err < min) {
    min <- fastfoodlover_err
    best_fit <- 'fastfoodlover'
  }

  return (best_fit)

}
```

Application and Usage

In the end, Diet Analyzer is an application with great ease of use, interactivity and information. The Meal selector is the main widget that comes into play; it lets the user easily select the meal they had during the day by choosing one food item at a time. Once a food item is selected,

(eg. Chicken, broiler, light meat and skin, roasted), you will be able to choose the quantity of it that you ate, from the ones available in the data set (eg. 1/2 bird, or 150ml chopped or diced). From here, you can add the food item to your meal and it displays as a table easy for the user to understand. If you make a mistake, you can easily remove it as well.

Once your meal is finalized, you can press the 'Analyze your meal button', which behind the scenes will calculate the amount of protein, fats and carbohydrates that your meal consumes depending of how much of each thing you ate. The app here will produce a pie chart of the proportions of these nutrients in your meal, as well as running a least squares error algorithm to find a best fit out of the baseline diets we provide for the user. The user will then be able to choose one of the baseline diets to compare their nutrient intake with the average person in that diet in a bar chart, to give them a clear view on whether they should increase or decrease their consumption of a certain nutrient.

Example

Next up we present an example of how a user will use our application with a sample meal we chose and get informative outputs. The methods used in this section, as well as in our shiny app, can be seen in the Appendix section of this report.

Let's suppose that our users meal consists of beef pot roast, with browned potatoes, peas and corn, fried chicken, and a cheese souffle as a dessert. In the shiny app the user will select the foods that his meal consisted of as well the specific quantities (volume) of each food and add them on a table like the one below.

```
# This meal would come from the search bar and the user's input. We are using a random mea
Meal <- FoodNames |>
  filter(food_description == 'Cheese souffle' | food_description == 'Beef pot roast, with
  inner_join(ConversionFactor, by = 'food_id') |>
  inner_join(MeasureNames, by = 'measure_id')  |>
  filter(measure_description == '250ml' | measure_description == '140g' | measure_descript

Meal |> select(food_description, measure_description)
```

```
# A tibble: 3 x 2
  food_description                                    measure_description
  <chr>                                               <chr>
1 Cheese souffle                                      250ml
2 Beef pot roast, with browned potatoes, peas and corn 140g
3 Chicken, broiler, light meat, fried                 100ml chopped or diced
```

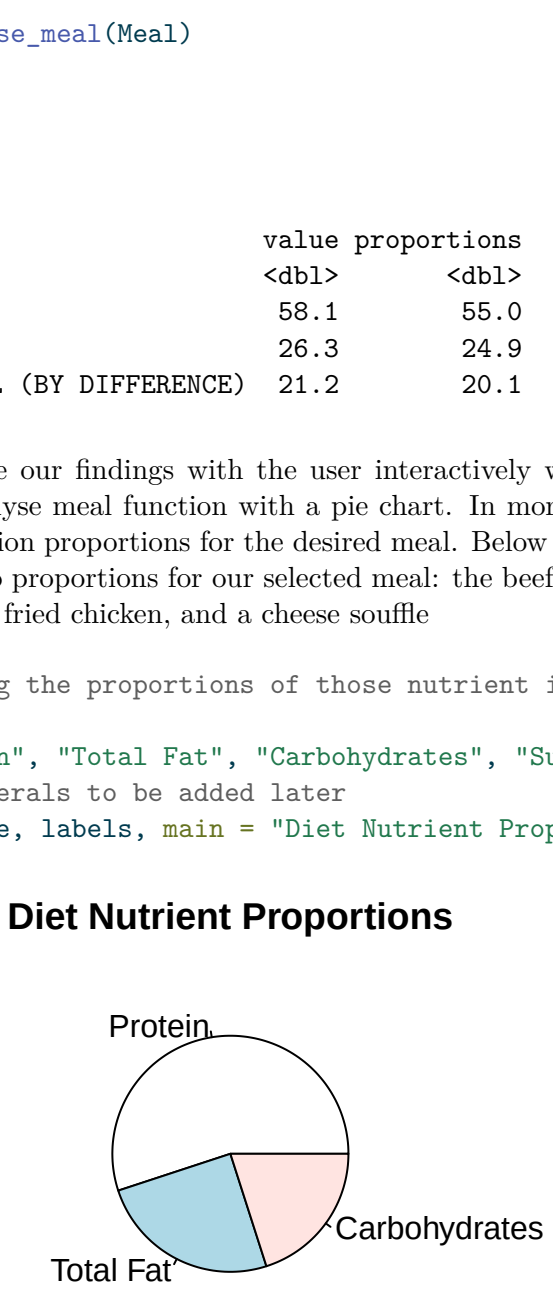
Using the analyse meal function (explain in the appendix) we will calculate the macros of the users meal. Specifically, the function would print the amount of protein, fats, and carbohydrates in the meal as well as what proportion of the total meal each macro was. In this particular example, the meal included 58g of protein, 26g of fat, and 21g of carbohydrates. Regarding proportions, the meal would consist of 55% protein, almost 25% fats and 20% carbohydrates.

```
Nutrients <- analyse_meal(Meal)

Nutrients
```

```
# A tibble: 3 x 3
  nutrient_name                       value proportions
  <chr>                               <dbl>       <dbl>
1 PROTEIN                              58.1        55.0
2 FAT (TOTAL LIPIDS)                   26.3        24.9
3 CARBOHYDRATE, TOTAL (BY DIFFERENCE)  21.2        20.1
```

In order to communicate our findings with the user interactively we are going to visualize the results from the analyse meal function with a pie chart. In more detail, we are going to showcase the meal nutrition proportions for the desired meal. Below you can see the pie chart DISPLAYING the macro proportions for our selected meal: the beef pot roast, with browned potatoes, peas and corn, fried chicken, and a cheese souffle

```
# Pie chart showing the proportions of those nutrient in our meal.

labels = c("Protein", "Total Fat", "Carbohydrates", "Sugars")
# Vitamins and minerals to be added later
pie(Nutrients$value, labels, main = "Diet Nutrient Proportions")
```



To provide additional information about the diversity of the user's diet we are also going to display the food group categories of the meal. This will be done by the food_group_prop function explained in the appendix. In our example, the fried chicken is considered a poultry product while the beef pot roast, with browned potatoes, peas and corn and a cheese souffle are in the mixed dishes category.

```
OurFoodGroups <- food_group_prop(Meal)

OurFoodGroups
```

```
# A tibble: 2 x 2
  food_group_name  `Number of items`
  <chr>                        <int>
1 Mixed Dishes                     2
2 Poultry Products                 1
```

Concluding, we would compare the users meal to different baseline diets (for example athlete, vegan, normal etc.) and using the least squares optimization problem find the baseline diet that is more similar to the users meal. Since in this example the suggested meal was high in protein and a similar amount of carbohydrates and fat the user's meal would be closer to the carnivore diet. This would be particularly helpful for people with nutrition goals that aim to follow a specific baseline diet.

```
best_fit <- find_diet_fit(Nutrients, athlete, normal, vegan, dessertlover, carnivore, fast

best_fit
```

```
[1] "carnivore"
```

Evaluation

We believe that our work is very valuable regarding a person trying to take care of their health, or physique, or even to learn what certain types of diets tend to look like in terms of protein, carbohydrates and fats. It is very easy to choose your own meal and the interactivity as well as the amount of foos options and meal combinations really makes it viable to use in a real life scenario and to quickly calculate your meal food group proportions.

As a target audience, we refer to those people trying to improve their eating habits or keep them stable if they are happy with their diet and physique. If you like data visualization or are a health nut, or both, you might love to use our Diet Analyzer to test different diets or meals. Even if you are a cook and want to try different variations of a dish and see which one

you think is more healthy, or as a consumer looking into the future to see what you want to buy at the grocery store for tomorrow's dinner.

Conclusion

In essence, the data package was used exhaustively and we fulfilled the goals that the (**TheNutrientResearchDivisionAtHealthCanada?**) intended when compiling this data set. As stated into their original documentation, the main token this package has is the measure and conversion factor tables that allow you to calculate the nutrient value of foods for different quantities eaten, and we exploited this advantage massively for creating our application. Not only can oyu interactively choose foods and discover their nutrient contents, but you can combine many of these to make your own meal in seconds, and compare it visually to other known diets to improve your own eating habits. The goals that we presented before completing our project were definitely met and the accessibility of the Shiny API was perfect to add the interactive component that we were seeking for. Now it is your turn to explore your eating habits, look into your family renowned recipes or plan ahead into a balanced lifestyle.

References

::: The Nutrient Research Division, Food Directoriate, Health Products and Food Branch, Health Canada :::

Appendix

```
#baseline diets
athlete <- tibble(
  nutrient_name = c("PROTEIN", "FAT (TOTAL LIPIDS)", "CARBOHYDRATE, TOTAL (BY DIFFERENCE)"
  proportions = c(20.0, 30.0, 50.0)
)


normal <- tibble(
  nutrient_name = c("PROTEIN", "FAT (TOTAL LIPIDS)", "CARBOHYDRATE, TOTAL (BY DIFFERENCE)"
  proportions = c(15.0, 27.5, 57.5)
)

vegan <- tibble(
  nutrient_name = c("PROTEIN", "FAT (TOTAL LIPIDS)", "CARBOHYDRATE, TOTAL (BY DIFFERENCE)"
  proportions = c(17.5, 27.5, 55.0)
)

fastfoodlover<- tibble(
  nutrient_name = c("PROTEIN", "FAT (TOTAL LIPIDS)", "CARBOHYDRATE, TOTAL (BY DIFFERENCE)"
  proportions = c(15.0, 37.5, 47.5)
)

carnivore <- tibble(
  nutrient_name = c("PROTEIN", "FAT (TOTAL LIPIDS)", "CARBOHYDRATE, TOTAL (BY DIFFERENCE)"
  proportions = c(30, 52.5, 17.5)
)

dessertlover <- tibble(
  nutrient_name = c("PROTEIN", "FAT (TOTAL LIPIDS)", "CARBOHYDRATE, TOTAL (BY DIFFERENCE)"
  proportions = c(7.5, 32.5, 60.0)
)

athlete
```

```
# A tibble: 3 x 2
  nutrient_name                       proportions
  <chr>                                     <dbl>
1 PROTEIN                                      20
2 FAT (TOTAL LIPIDS)                           30
3 CARBOHYDRATE, TOTAL (BY DIFFERENCE)          50
```

```
vegan
```

```
# A tibble: 3 x 2
  nutrient_name                       proportions
  <chr>                                     <dbl>
1 PROTEIN                                    17.5
2 FAT (TOTAL LIPIDS)                         27.5
3 CARBOHYDRATE, TOTAL (BY DIFFERENCE)        55
```

```
normal
```

```
# A tibble: 3 x 2
  nutrient_name                       proportions
  <chr>                                     <dbl>
1 PROTEIN                                    15
2 FAT (TOTAL LIPIDS)                         27.5
3 CARBOHYDRATE, TOTAL (BY DIFFERENCE)        57.5
```

```
dessertlover
```

```
# A tibble: 3 x 2
  nutrient_name                       proportions
  <chr>                                     <dbl>
1 PROTEIN                                     7.5
2 FAT (TOTAL LIPIDS)                         32.5
3 CARBOHYDRATE, TOTAL (BY DIFFERENCE)        60
```

```
carnivore
```

```
# A tibble: 3 x 2
  nutrient_name                          proportions
  <chr>                                        <dbl>
1 PROTEIN                                       30
2 FAT (TOTAL LIPIDS)                            52.5
3 CARBOHYDRATE, TOTAL (BY DIFFERENCE)           17.5
```

```
fastfoodlover
```

```
# A tibble: 3 x 2
  nutrient_name                          proportions
  <chr>                                        <dbl>
1 PROTEIN                                       15
2 FAT (TOTAL LIPIDS)                            37.5
3 CARBOHYDRATE, TOTAL (BY DIFFERENCE)           47.5
```

```
sum_squared_errors <- function(props, baseline) {
  props |> inner_join(baseline, by = 'nutrient_name') |>
    mutate(sq_error = (proportions.x - proportions.y)^2) |>
    summarise(sum_squared_errors = sum(sq_error))
}
```

```
find_diet_fit <- function(props, athlete, normal, vegan, dessertlover, carnivore, fastfood

  best_fit <- 'athlete'
  athlete_err <- sum_squared_errors(props, athlete)

  min <- athlete_err

  normal_err <- sum_squared_errors(props, normal)
  if (normal_err < min) {
    min <- normal_err
    best_fit <- 'normal'
  }

  vegan_err <- sum_squared_errors(props, vegan)
  if (vegan_err < min) {
    min <- vegan_err
    best_fit <- 'vegan'
  }
```

```
  dessertlover_err <- sum_squared_errors(props, dessertlover)
  if (dessertlover_err < min) {
    min <- dessertlover_err
    best_fit <- 'dessertlover'
  }

  carnivore_err <- sum_squared_errors(props, carnivore)
  if (carnivore_err < min) {
    min <- carnivore_err
    best_fit <- 'carnivore'
  }

  fastfoodlover_err <- sum_squared_errors(props, fastfoodlover)
  if (fastfoodlover_err < min) {
    min <- fastfoodlover_err
    best_fit <- 'fastfoodlover'
  }

  return (best_fit)

}
```

```
# To help us quantify nutrients based on how much of the food the user ate.
quantify_nutrient <- function(food, nutrient) {
  myprops <- NutrientNames |>
    inner_join(NutrientAmounts, by = 'nutrient_id') |>
    rename("vitamin_b12_in_g" = nutrient_value) |>
    filter(nutrient_name == nutrient) |>
    inner_join(FoodNames, by = 'food_id') |>
    filter(food_description == food) |>
    inner_join(ConversionFactor, by = 'food_id') |>
    inner_join(MeasureNames, by = 'measure_id') |>
    mutate(total_vitamin_b12_in_g = vitamin_b12_in_g * conversion_factor_value) |>
    select(measure_description, conversion_factor_value, total_vitamin_b12_in_g)
  return (myprops)
}
```

```
# Wrangling to get the value of each nutrient for our meal and select the nutrients that w

analyse_meal <- function(Meal) {
```

```
  Nutrients <- Meal |>
  inner_join(NutrientAmounts, by = 'food_id') |>
  inner_join(NutrientNames, by = 'nutrient_id') |>
  filter(nutrient_name == 'PROTEIN' | nutrient_name == 'FAT (TOTAL LIPIDS)' | nutrient_nam
  mutate(new_value = nutrient_value * conversion_factor_value) |>
  group_by(nutrient_name) |>
  summarise("value" = sum(new_value)) |>
  mutate(proportions = value * 100 / sum(value)) |>
  select(nutrient_name, value, proportions) |>
  arrange(desc(value))

  return (Nutrients)
}
```

```
# Count items in each food group, will find a better way to quantify this
# Would look better with an actual meal rather than three items
food_group_prop <- function(meal) {
  OurFoodGroups <- meal |>
    inner_join(FoodGroup, by = 'food_group_id') |>
    group_by(food_group_name) |>
    summarise("Number of items" = n())
  return (OurFoodGroups)
}
```